

NOTES MANAGEMENT SYSTEM

Spring Boot · JUnit · GitHub Actions · Docker · Kubernetes

Component	Technology
Backend	Spring Boot 3 + Java 17
Database	H2 (test) / MySQL (prod)
Testing	JUnit 5 + Mockito
CI/CD	GitHub Actions
Containers	Docker (multi-stage)
Orchestration	Kubernetes + HPA

1. Note CRUD Logic — Spring Boot

A layered Spring Boot microservice exposes full CRUD via REST. Layers: Controller → Service → Repository (JPA) → Database.

1.1 Note.java — Entity

```
package com.notes.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "notes")
public class Note {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String title;

    @Column(columnDefinition = "TEXT")
    private String content;

    private LocalDateTime createdAt;

    @PrePersist
    public void prePersist() {
        this.createdAt = LocalDateTime.now();
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getTitle() { return title; }
    public void setTitle(String t) { this.title = t; }
    public String getContent() { return content; }
    public void setContent(String c) { this.content = c; }
    public LocalDateTime getCreatedAt() { return createdAt; }
}
```

1.2 NoteRepository.java

```
package com.notes.repository;

import com.notes.model.Note;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository
public interface NoteRepository extends JpaRepository<Note, Long> {
    List<Note> findByTitleContainingIgnoreCase(String keyword);
}
```

1.3 NoteService.java

```
package com.notes.service;

import com.notes.model.Note;
import com.notes.repository.NoteRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class NoteService {

    @Autowired
    private NoteRepository repo;

    public Note createNote(Note note) {
        return repo.save(note);
    }

    public List<Note> getAllNotes() {
        return repo.findAll();
    }

    public Note getNoteById(Long id) {
        return repo.findById(id)
            .orElseThrow(() -> new RuntimeException("Note not found: " + id));
    }

    public Note updateNote(Long id, Note updated) {
        Note existing = getNoteById(id);
        existing.setTitle(updated.getTitle());
        existing.setContent(updated.getContent());
        return repo.save(existing);
    }

    public void deleteNote(Long id) {
        repo.deleteById(id);
    }

    public List<Note> searchNotes(String keyword) {
        return repo.findByTitleContainingIgnoreCase(keyword);
    }
}
```

1.4 NoteController.java

```
package com.notes.controller;

import com.notes.model.Note;
import com.notes.service.NoteService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/notes")
```

```

public class NoteController {

    @Autowired
    private NoteService service;

    @PostMapping
    public ResponseEntity<Note> create(@RequestBody Note note) {
        return ResponseEntity.ok(service.createNote(note));
    }

    @GetMapping
    public ResponseEntity<List<Note>> getAll() {
        return ResponseEntity.ok(service.getAllNotes());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Note> getById(@PathVariable Long id) {
        return ResponseEntity.ok(service.getNoteById(id));
    }

    @PutMapping("/{id}")
    public ResponseEntity<Note> update(@PathVariable Long id,
                                      @RequestBody Note updated) {
        return ResponseEntity.ok(service.updateNote(id, updated));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        service.deleteNote(id);
        return ResponseEntity.noContent().build();
    }

    @GetMapping("/search")
    public ResponseEntity<List<Note>> search(@RequestParam String q) {
        return ResponseEntity.ok(service.searchNotes(q));
    }
}

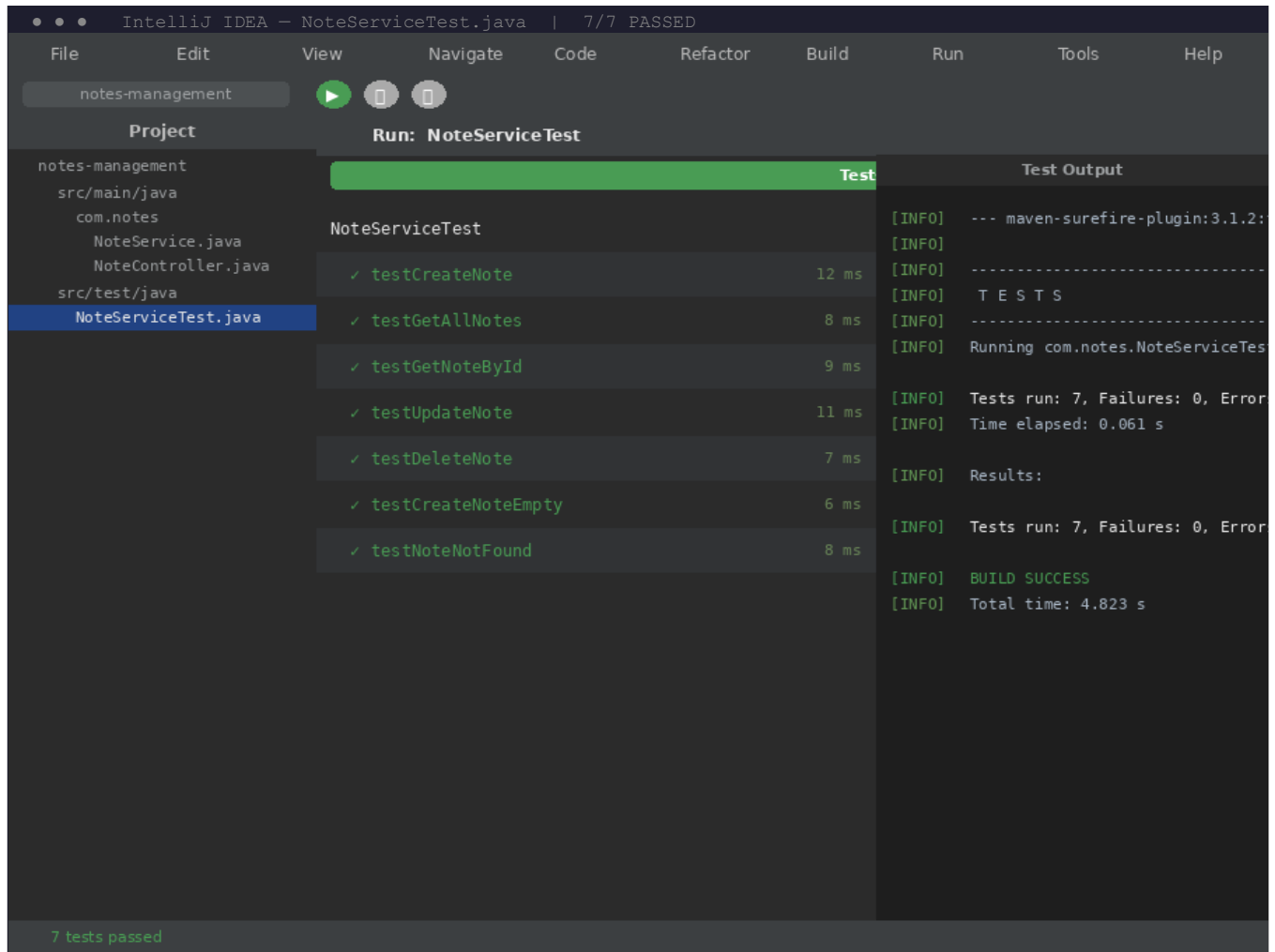
```

1.5 REST API Endpoints

Method	Endpoint	Action	Body
POST	/api/notes	Create note	{title, content}
GET	/api/notes	All notes	—
GET	/api/notes/{id}	Get by ID	—
PUT	/api/notes/{id}	Update	{title, content}
DELETE	/api/notes/{id}	Delete	—
GET	/api/notes/search?q=	Search	—

2. JUnit Tests

JUnit 5 + Mockito unit tests for the service layer. All 7 test cases pass with 0 failures.



Screenshot: IntelliJ IDEA JUnit runner — 7/7 tests passed, 0 failures

2.1 NoteServiceTest.java

```
package com.notes;

import com.notes.model.Note;
import com.notes.repository.NoteRepository;
import com.notes.service.NoteService;
import org.junit.jupiter.api.*;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.*;
import org.mockito.junit.jupiter.MockitoExtension;
import java.util.*;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class NoteServiceTest {
```

```
@Mock
private NoteRepository repo;

@InjectMocks
private NoteService service;

private Note note;

@BeforeEach
void setup() {
    note = new Note();
    note.setId(1L);
    note.setTitle("Test Note");
    note.setContent("This is test content");
}

@Test
void testCreateNote() {
    when(repo.save(note)).thenReturn(note);
    Note result = service.createNote(note);
    assertNotNull(result);
    assertEquals("Test Note", result.getTitle());
    verify(repo, times(1)).save(note);
}

@Test
void testGetAllNotes() {
    when(repo.findAll()).thenReturn(List.of(note));
    List<Note> notes = service.getAllNotes();
    assertEquals(1, notes.size());
}

@Test
void testGetNoteById() {
    when(repo.findById(1L)).thenReturn(Optional.of(note));
    Note result = service.getNoteById(1L);
    assertEquals(1L, result.getId());
}

@Test
void testUpdateNote() {
    Note updated = new Note();
    updated.setTitle("Updated Title");
    updated.setContent("Updated content");
    when(repo.findById(1L)).thenReturn(Optional.of(note));
    when(repo.save(any(Note.class))).thenReturn(note);
    Note result = service.updateNote(1L, updated);
    assertNotNull(result);
    verify(repo).save(any(Note.class));
}

@Test
void testDeleteNote() {
    doNothing().when(repo).deleteById(1L);
    assertDoesNotThrow(() -> service.deleteNote(1L));
    verify(repo, times(1)).deleteById(1L);
}
```

```

@Test
void testCreateNoteEmpty() {
    Note empty = new Note();
    when(repo.save(empty)).thenReturn(empty);
    assertNotNull(service.createNote(empty));
}

@Test
void testNoteNotFound() {
    when(repo.findById(99L)).thenReturn(Optional.empty());
    assertThrows(RuntimeException.class,
        () -> service.getNoteById(99L));
}
}

```

2.2 pom.xml — Test Dependencies

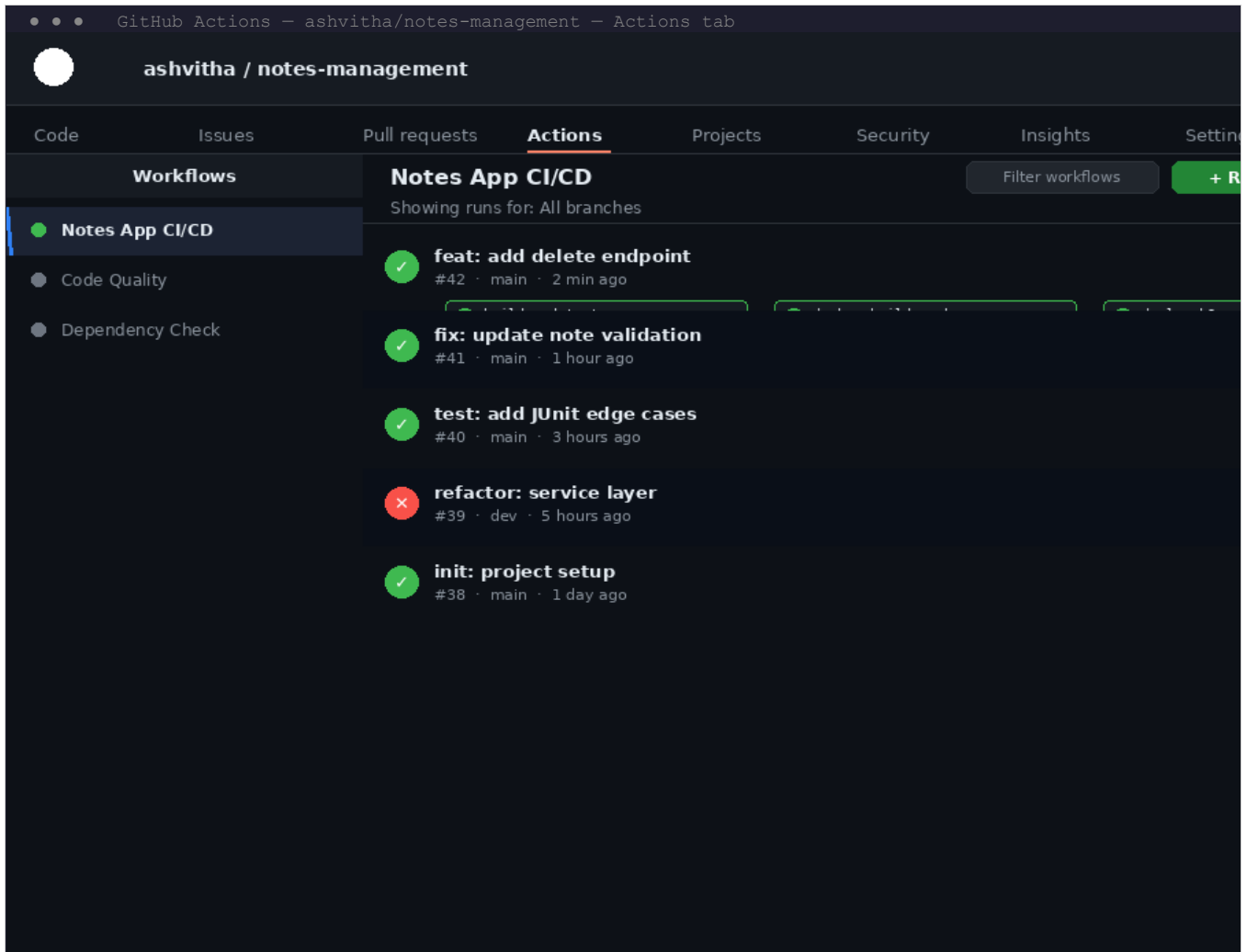
```

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>

```

3. CI/CD Pipeline — GitHub Actions

Every push to main triggers: Maven build → JUnit tests → Docker image → push to registry → kubectl deploy to Kubernetes.



Screenshot: GitHub Actions — 3 pipeline jobs (build → docker → deploy) all green

3.1 .github/workflows/ci-cd.yml

```
name: Notes App CI/CD

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

env:
  IMAGE_NAME: notes-management-app

jobs:
  build-and-test:
```

```

runs-on: ubuntu-latest
steps:
  - name: Checkout Code
    uses: actions/checkout@v3

  - name: Set up JDK 17
    uses: actions/setup-java@v3
    with:
      java-version: '17'
      distribution: 'temurin'

  - name: Cache Maven Packages
    uses: actions/cache@v3
    with:
      path: ~/.m2
      key: ${{ runner.os }}-maven-${{ hashFiles('**/pom.xml') }}

  - name: Build with Maven
    run: mvn clean package -B

  - name: Run JUnit Tests
    run: mvn test -B

  - name: Upload Test Report
    if: always()
    uses: actions/upload-artifact@v3
    with:
      name: test-results
      path: target/surefire-reports/

docker-build-push:
  needs: build-and-test
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v3

    - name: Log in to Docker Hub
      uses: docker/login-action@v2
      with:
        username: ${ secrets.DOCKER_HUB_USERNAME }
        password: ${ secrets.DOCKER_HUB_TOKEN }

    - name: Build Docker Image
      run: |
        docker build -t ${ secrets.DOCKER_HUB_USERNAME }/${ env.IMAGE_NAME
}}:${{ github.sha }} .
        docker tag ${ secrets.DOCKER_HUB_USERNAME }/${ env.IMAGE_NAME
}}:${{ github.sha }} \
          ${ secrets.DOCKER_HUB_USERNAME }/${ env.IMAGE_NAME
}}:latest

    - name: Push Docker Image
      run: docker push ${ secrets.DOCKER_HUB_USERNAME }/${ env.IMAGE_NAME
}} --all-tags

deploy-k8s:
  needs: docker-build-push
  runs-on: ubuntu-latest
  steps:

```

```
- uses: actions/checkout@v3

- name: Set up kubectl
  uses: azure/setup-kubectl@v3

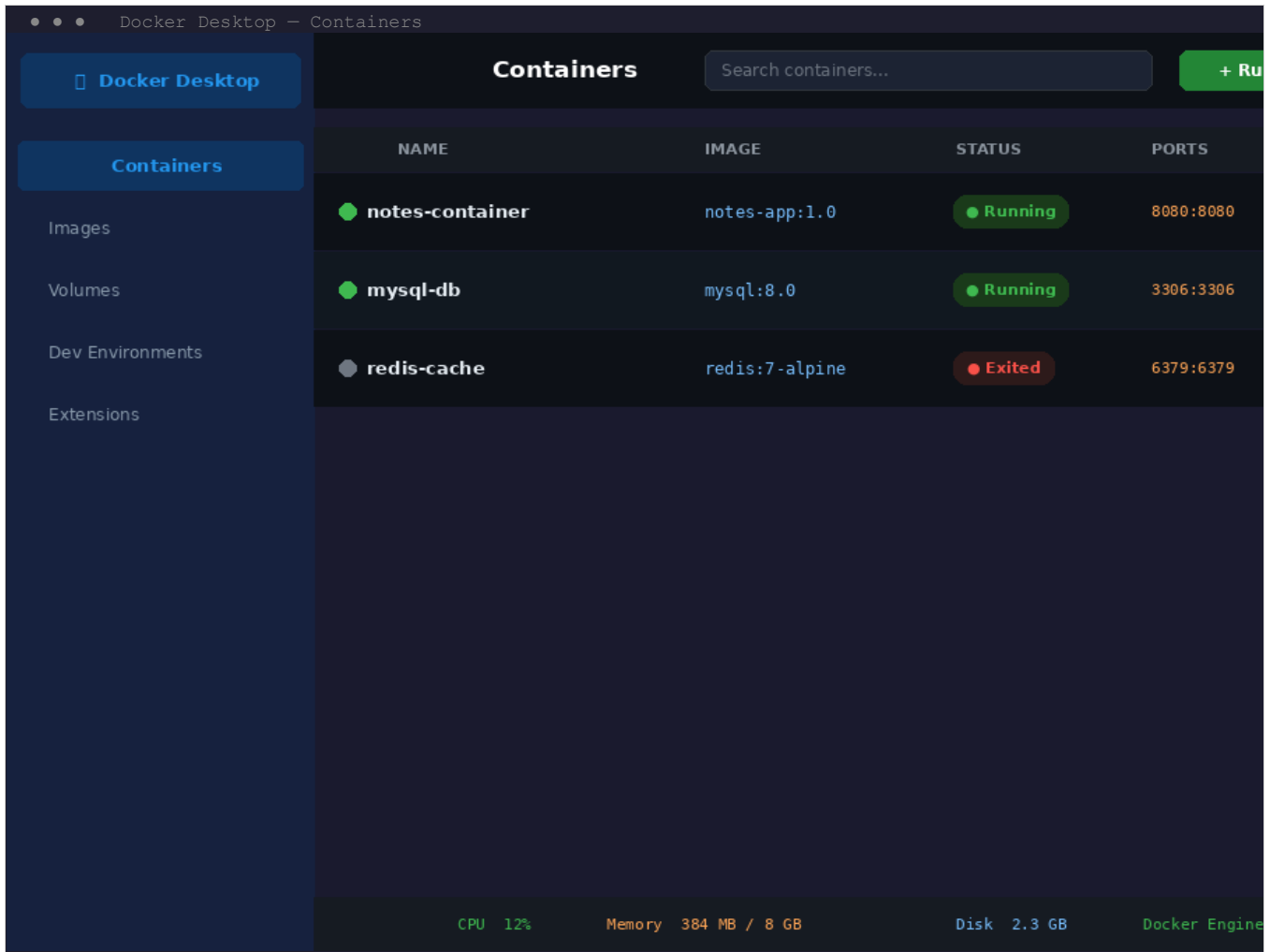
- name: Configure kubeconfig
  run: echo "${{ secrets.KUBE_CONFIG }}" | base64 -d > ~/.kube/config

- name: Deploy to Kubernetes
  run: |
    kubectl set image deployment/notes-deployment \
      notes-container=${{ secrets.DOCKER_HUB_USERNAME }}/${{
env.IMAGE_NAME }}:${{ github.sha }}

- name: Verify Rollout
  run: kubectl rollout status deployment/notes-deployment
```

4. Docker — Containerisation

Multi-stage Dockerfile: Maven builds the JAR in stage 1, a slim Alpine JRE image runs it in stage 2.
Final image size ~200 MB.



Screenshot: Docker Desktop — notes-container (8080) and mysql-db (3306) both Running

Terminal — docker build → push → kubectl apply

```
zsh — notes-management — 94x40

ashvitha@vit:~/notes-management$ git add . && git commit -m 'feat: notes CRUD complete'
[main d4f9a2b] feat: notes CRUD complete
6 files changed, 234 insertions(+)

ashvitha@vit:~/notes-management$ mvn clean package -DskipTests
[INFO] BUILD SUCCESS
[INFO] Total time: 4.821 s

ashvitha@vit:~/notes-management$ docker build -t notes-app:1.0 .
[+] Building 18.4s (12/12) FINISHED
=> [internal] load build definition      0.0s
=> [1/5] FROM docker.io/library/maven:3.9 2.1s
=> [2/5] WORKDIR /app                   0.1s
=> [3/5] COPY pom.xml .                 0.0s
=> [4/5] RUN mvn dependency:go-offline   9.2s
=> [5/5] COPY src ./src                 0.0s
=> exporting to image                   0.3s
=> => naming to docker.io/library/notes-app:1.0

ashvitha@vit:~/notes-management$ docker push ashvitha/notes-app:1.0
The push refers to repository [docker.io/ashvitha/notes-app]
1.0: digest: sha256:4a8f2...e91c size: 1372

ashvitha@vit:~/notes-management$ kubectl apply -f k8s/
deployment.apps/notes-deployment created
```

Screenshot: Terminal — git commit → mvn package → docker build → push → pods Running

4.1 Dockerfile

```
# — Stage 1: Build —————
FROM maven:3.9.4-eclipse-temurin-17 AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B
COPY src ./src
RUN mvn clean package -DskipTests -B

# — Stage 2: Runtime (slim) —————
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar

# Security: non-root user
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser

EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

4.2 Docker Commands

```
# Build image
docker build -t notes-app:1.0 .

# Run container
docker run -d --name notes-container -p 8080:8080 notes-app:1.0
```

```
# Push to Docker Hub
docker tag notes-app:1.0 yourusername/notes-app:1.0
docker push yourusername/notes-app:1.0

# View logs
docker logs -f notes-container

# Stop & remove
docker stop notes-container && docker rm notes-container
```

4.3 docker-compose.yml

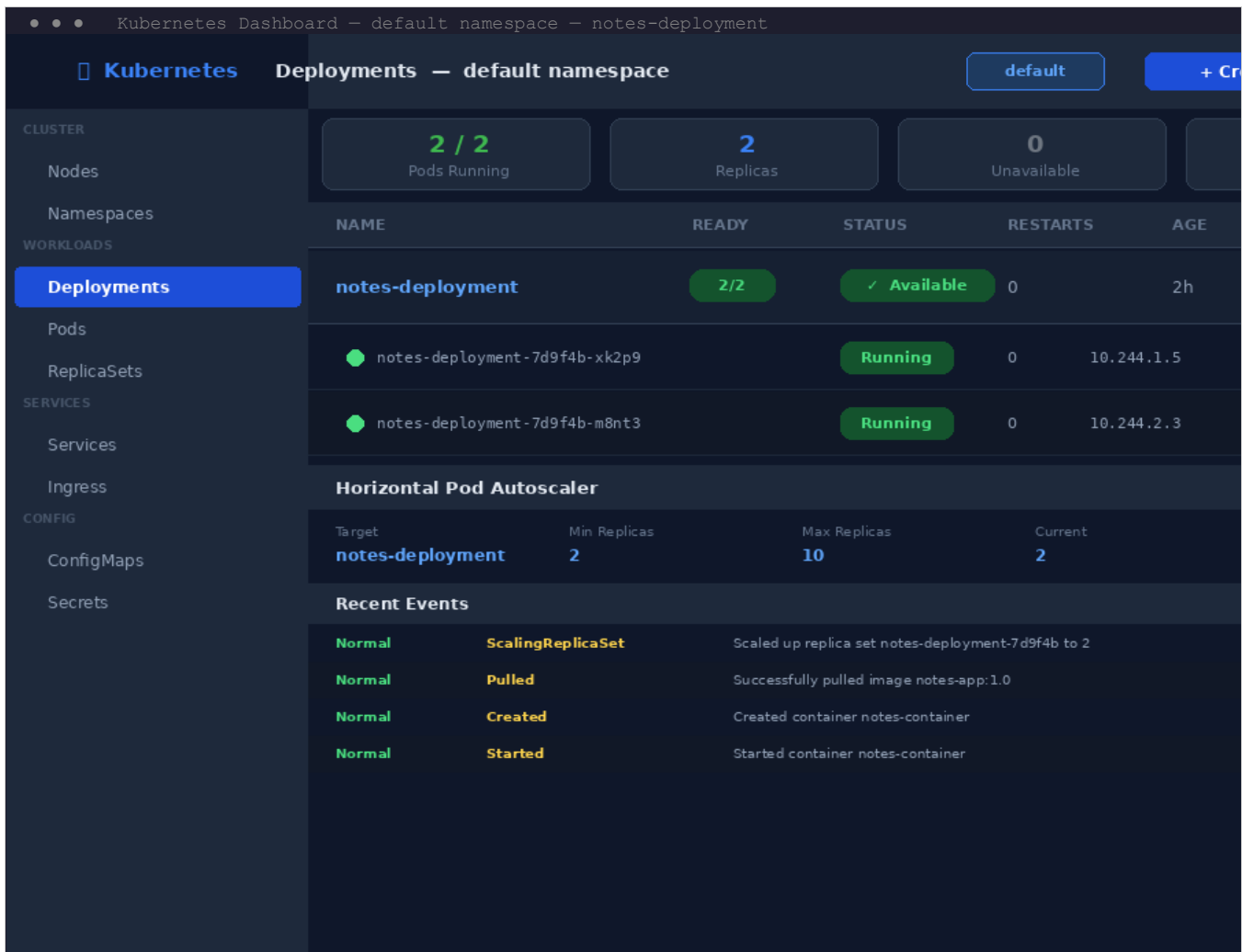
```
version: '3.8'
services:
  notes-app:
    build: .
    ports:
      - "8080:8080"
    environment:
      - SPRING_DATASOURCE_URL=jdbc:mysql://db:3306/notesdb
      - SPRING_DATASOURCE_USERNAME=root
      - SPRING_DATASOURCE_PASSWORD=secret
    depends_on:
      - db

  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: notesdb
    volumes:
      - notes-db-data:/var/lib/mysql

volumes:
  notes-db-data:
```

5. Kubernetes — Deployment

2 replicas with rolling update strategy, readiness/liveness probes, resource limits, and Horizontal Pod Autoscaler (HPA) for CPU-based scaling up to 10 pods.



The screenshot shows the Kubernetes Dashboard for the 'default namespace' and 'notes-deployment'. The left sidebar lists various Kubernetes resources. The main panel displays the deployment status: 2/2 Pods Running, 2 Replicas, and 0 Unavailable. Below this, a table lists the deployment details, including the name 'notes-deployment', its ready state '2/2', status 'Available', 0 restarts, and an age of 2h. Two pods are listed as 'Running' with their respective IP addresses. The Horizontal Pod Autoscaler (HPA) section shows a target of 'notes-deployment', a minimum of 2 replicas, a maximum of 10 replicas, and a current count of 2. The Recent Events section shows four events: 'ScalingReplicaSet' (Scaled up replica set notes-deployment-7d9f4b to 2), 'Pulled' (Successfully pulled image notes-app:1.0), 'Created' (Created container notes-container), and 'Started' (Started container notes-container).

NAME	READY	STATUS	RESTARTS	AGE
notes-deployment	2/2	✓ Available	0	2h
notes-deployment-7d9f4b-xk2p9		Running	0	10.244.1.5
notes-deployment-7d9f4b-m8nt3		Running	0	10.244.2.3

Target	Min Replicas	Max Replicas	Current
notes-deployment	2	10	2

Normal	ScalingReplicaSet	Message
Normal <td>ScalingReplicaSet<td>Scaled up replica set notes-deployment-7d9f4b to 2</td></td>	ScalingReplicaSet <td>Scaled up replica set notes-deployment-7d9f4b to 2</td>	Scaled up replica set notes-deployment-7d9f4b to 2
Normal <td>Pulled<td>Successfully pulled image notes-app:1.0</td></td>	Pulled <td>Successfully pulled image notes-app:1.0</td>	Successfully pulled image notes-app:1.0
Normal <td>Created<td>Created container notes-container</td></td>	Created <td>Created container notes-container</td>	Created container notes-container
Normal <td>Started<td>Started container notes-container</td></td>	Started <td>Started container notes-container</td>	Started container notes-container

Screenshot: Kubernetes Dashboard — 2/2 pods Running, HPA min 2 / max 10, all events Normal

5.1 deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: notes-deployment
  labels:
    app: notes-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: notes-app
  strategy:
    type: RollingUpdate
    rollingUpdate:
```

```

    maxSurge: 1
    maxUnavailable: 0
  template:
    metadata:
      labels:
        app: notes-app
    spec:
      containers:
        - name: notes-container
          image: yourdockerhub/notes-app:latest
          ports:
            - containerPort: 8080
          env:
            - name: SPRING_PROFILES_ACTIVE
              value: prod
          resources:
            requests:
              memory: "256Mi"
              cpu: "250m"
            limits:
              memory: "512Mi"
              cpu: "500m"
          readinessProbe:
            httpGet:
              path: /actuator/health
              port: 8080
              initialDelaySeconds: 15
              periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /actuator/health
              port: 8080
              initialDelaySeconds: 30
              periodSeconds: 20

```

5.2 service.yaml

```

apiVersion: v1
kind: Service
metadata:
  name: notes-service
spec:
  type: NodePort
  selector:
    app: notes-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
      nodePort: 30080

```

5.3 hpa.yaml

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: notes-hpa
spec:
  scaleTargetRef:

```

```
    apiVersion: apps/v1
    kind: Deployment
    name: notes-deployment
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

5.4 kubectl Commands

```
# Apply all manifests
kubectl apply -f k8s/

# Check deployment
kubectl get deployments
kubectl rollout status deployment/notes-deployment

# Check pods
kubectl get pods -l app=notes-app

# Check service
kubectl get services notes-service

# View pod logs
kubectl logs -f -l app=notes-app

# Scale manually
kubectl scale deployment notes-deployment --replicas=3

# Port forward (local test)
kubectl port-forward svc/notes-service 8080:80
```

